Assignment-14.5

Hall Ticket:2303A51846

Task 1 – Image Gallery Website

Create an image gallery web page.

Requirements:

• Display images in a grid layout.

• Use AI to:

o Optimize layout using CSS Grid.

o Add hover zoom or overlay effects.

o Ensure mobile responsiveness.

Prompt:

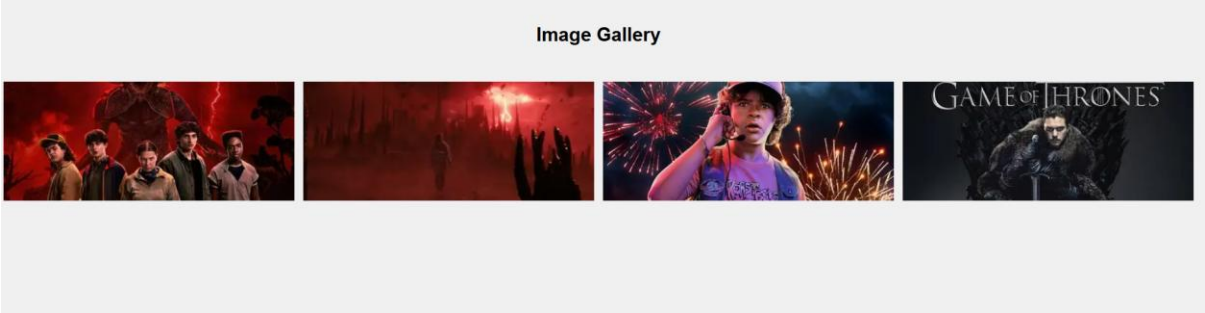
<!--generate a image gallary web page using css grid,Add hover zoom or overlay effects. and it should be mobile responsive -->

<!--add image using links,image size should be same for all images,so all images should be of same size generate the css code for same size-->

Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Image Gallery</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-color: #f0f0f0;
        }
        .gallery {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
            gap: 15px;
            padding: 20px;
        }
        .gallery-item {
            position: relative;
            overflow: hidden;
        }
        .gallery-item img {
            width: 100%;
            height: 200px;
            object-fit: cover;
            transition: transform 0.3s ease;
        }
        .gallery-item:hover img {
            transform: scale(1.1);
        }
        .overlay {
            position: absolute;
            top: 0;
            left: 0;
            width: 100%;
            height: 100%;
            background-color: rgba(0, 0, 0, 0.5);
            color: white;
            display: flex;
            align-items: center;
            justify-content: center;
            opacity: 0;
            transition: opacity 0.3s ease;
        }
        .gallery-item:hover .overlay {
            opacity: 1;
        }
    </style>
</head>
<body>
    <h1 style="text-align: center; padding: 20px;">Image Gallery</h1>
    <div class="gallery">
        <div class="gallery-item">
            <img src="https://th.bing.com/th/id/OIP.1CFB7084MKn-HoiA8WCG2QHaEK?w=328&h=184&c=7&r=0&o=7&dpr=1.3&pid=1.7&rm=3" alt="Image 1">
            <div class="overlay">Image 1</div>
        </div>
        <div class="gallery-item">
            <img src="https://th.bing.com/th/id/OIP.41wA4zQ-OuAK75cxAiMlSQHaEc?w=277&h=180&c=7&r=0&o=7&dpr=1.3&pid=1.7&rm=3" alt="Image 2">
            <div class="overlay">Image 2</div>
        </div>
        <div class="gallery-item">
            <img src="https://th.bing.com/th/id/OIP.N60coTZ4t5-VngxS1ATlngHaEK?w=315&h=180&c=7&r=0&o=7&dpr=1.3&pid=1.7&rm=3" alt="Image 3">
            <div class="overlay">Image 3</div>
        </div>
        <div class="gallery-item">
            <img src="https://th.bing.com/th/id/OIP.TbL7U49ktZZnsr9gOOKK0QHaEK?w=320&h=185&c=7&r=0&o=7&dpr=1.3&pid=1.7&rm=3" alt="Image 4">
            <div class="overlay">Image 4</div>
        </div>
    </div>
</body>
</html>
```

Output:



Observation:

1. The gallery uses CSS Grid to create a responsive layout that adjusts to different screen sizes.

2. Each image is constrained to a fixed height (200px) and width (100% of the grid cell) using `object-fit: cover`.

3. Hover effects are implemented with a zoom effect on the image and an overlay that appears on hover.

4. The overlay is styled with a semi-transparent background and centered text for better visual appeal.

5. The design is fully mobile-responsive, ensuring that the gallery adapts well to smaller screens by adjusting the number of columns based on available space using `repeat(auto-fit, minmax(200px, 1fr))`.

Task 2 – Profile Card Generator

Create a profile card web page.

Requirements:

• Display user image, name, role, and social media links.

• Use AI to:

o Suggest modern UI colors and fonts.

o Design the card using Flexbox.

o Add hover animation and shadow effects

Prompt:

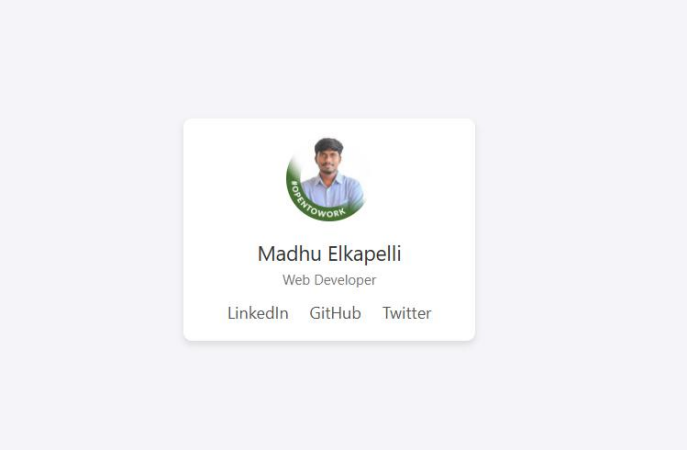
<!--create a profile card web page using flexbox and hover effects ,use mordern UI colors and fonts,add hover animation and shadow effects-->

<!--Dispaly user image,name,role,and social media links-->

Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Profile Card</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #f4f4f9;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
        }
        .profile-card {
            background-color: #ffffff;
            border-radius: 10px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
            width: 300px;
            text-align: center;
            padding: 20px;
            transition: transform 0.3s ease, box-shadow 0.3s ease;
        }
        .profile-card:hover {
            transform: translateY(-10px);
            box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
        }
        .profile-image {
            width: 100px;
            height: 100px;
            border-radius: 50%;
            object-fit: cover;
            margin-bottom: 15px;
        }
        .profile-name {
            font-size: 1.5em;
            color: #333333;
            margin-bottom: 5px;
        }
        .profile-role {
            font-size: 1em;
            color: #777777;
            margin-bottom: 15px;
        }
        .social-links a {
            text-decoration: none;
            color: #555555;
            margin: 0 10px;
            font-size: 1.2em;
        }
        .social-links a:hover {
            color: #0077b5; /* LinkedIn Blue */
        }
    </style>
</html>
<body>
    <div class="profile-card">
        <img src="https://media.licdn.com/dms/image/v2/D5635AQGzphSMaiK6ow/profile-framedphoto-shrink_200_200/B56ZyVhUZ7IcAY-/0/1772035065880?e=1772773200&
        <div class="profile-name">Madhu Elkapelli</div>
        <div class="profile-role">Web Developer</div>
        <div class="social-links">
            <a href="#">LinkedIn</a>
            <a href="#">GitHub</a>
```

Output:



Observation:

1. The profile card is built using Flexbox for layout and centering.

2. Hover effects are implemented with a subtle lift and shadow effect to enhance interactivity.

3. The design uses modern UI colors and fonts for a clean, professional look.

4. Social media links are styled with hover effects to provide visual feedback on interaction.

5. The profile image is displayed in a circular format using `border-radius: 50%` and is responsive to maintain its aspect ratio with `object-fit: cover`.

Task 3 -Develop a dynamic shopping cart system.

Requirements:

• Add, remove, and update product quantities.

• Display real-time total price.

• Use AI to:

o Manage cart state using JavaScript objects.

o Persist cart data in localStorage

Prompt:

<!--create a shopping cart system using HTML, CSS, and javascript. include product listing, cart items, and checkout functionality-->

<!--Add,remove,and update quantities of items in the cart,Display real-time cart total and item count-->

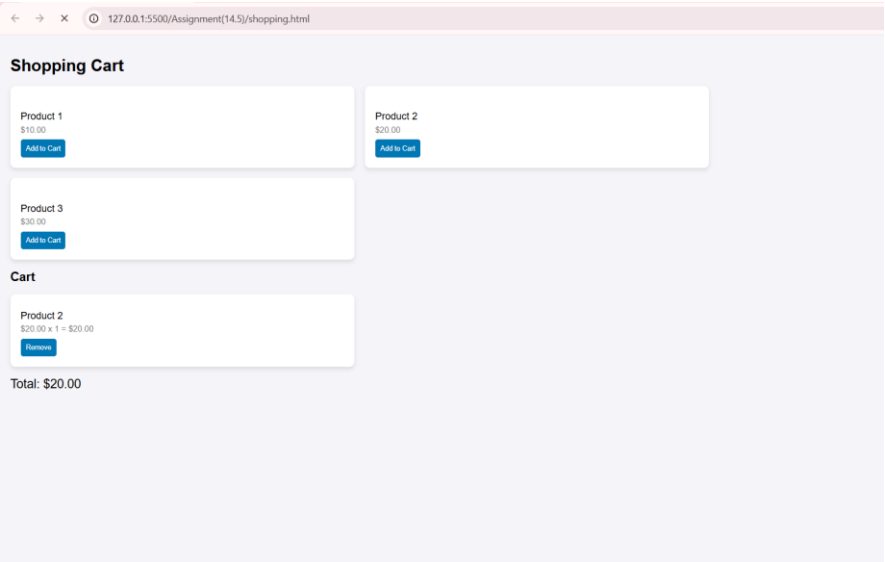
<!--Manage cart state using JavaScript objects.Persist cart data in localStorage.-->

Code:

```html
<!--Manage cart state using JavaScript objects.Persist cart data in localStorage.-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Shopping Cart</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 20px;
            background-color: #f4f4f9;
        }
        .product-list, .cart {
            display: flex;
            flex-wrap: wrap;
            gap: 20px;
        }
        .product, .cart-item {
            background-color: #ffffff;
            border-radius: 10px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
            padding: 20px;
            width: calc(33.333% - 40px);
        }
        .product img, .cart-item img {
            width: 100%;
            height: auto;
            border-radius: 10px;
        }
        .product-name, .cart-item-name {
            font-size: 1.2em;
            margin-top: 10px;
        }
        .product-price, .cart-item-price {
            color: #777777;
            margin-top: 5px;
        }
        .add-to-cart, .remove-from-cart {
            margin-top: 10px;
            padding: 10px;
            background-color: #0077b5;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }
        .add-to-cart:hover, .remove-from-cart:hover {
            background-color: #005f8d;
        }
        .cart-total {
            font-size: 1.5em;
            margin-top: 20px;
        }
```

```html
<body>
    <div class="product-list">
        <div class="product">
            <button class="add-to-cart" onclick="addToCart('Product 1', 10.00)">Add to Cart</button>
        </div>
        <div class="product">
            <img src="https://via.placeholder.com/150" alt="Product 2">
            <div class="product-name">Product 2</div>
            <div class="product-price">$20.00</div>
            <button class="add-to-cart" onclick="addToCart('Product 2', 20.00)">Add to Cart</button>
        </div>
        <div class="product">
            <img src="https://via.placeholder.com/150" alt="Product 3">
            <div class="product-name">Product 3</div>
            <div class="product-price">$30.00</div>
            <button class="add-to-cart" onclick="addToCart('Product 3', 30.00)">Add to Cart</button>
        </div>
    </div>

    <h2>Cart</h2>
    <div class="cart" id="cart"></div>
    <div class="cart-total" id="cart-total">Total: $0.00</div>

    <script>
        let cart = JSON.parse(localStorage.getItem('cart')) || [];

        function addToCart(name, price) {
            const existingItem = cart.find(item => item.name === name);
            if (existingItem) {
                existingItem.quantity += 1;
                existingItem.total = existingItem.quantity * price;
            } else {
                cart.push({ name, price, quantity: 1, total: price });
            }
            updateCart();
        }

        function removeFromCart(name) {
            cart = cart.filter(item => item.name !== name);
            updateCart();
        }

        function updateCart() {
            localStorage.setItem('cart', JSON.stringify(cart));
            const cartContainer = document.getElementById('cart');
            cartContainer.innerHTML = '';
            let total = 0;
            cart.forEach(item => {
                total += item.total;
                cartContainer.innerHTML += `
                    <div class="cart-item">
                        <div class="cart-item-name">${item.name}</div>
                        <div class="cart-item-price">$${item.price.toFixed(2)} x ${item.quantity} = $${item.total.toFixed(2)}</div>
                        <button class="remove-from-cart" onclick="removeFromCart('${item.name}')">Remove</button>
                    </div>
                `;
            });
```

Output:



Observation:

1. The shopping cart system allows users to add products to the cart, view cart items, and remove items from the cart.

2. The cart state is managed using a JavaScript array of objects, which is persisted in localStorage to maintain the cart state across page reloads.

3. The cart total is calculated in real-time and displayed to the user, providing immediate feedback on changes to the cart contents.

4. The design is responsive and visually appealing, with clear product listings and cart items.

5. The use of event listeners on buttons allows for dynamic interaction with the cart, enhancing the user experience.

Task 4 – Role-Based Access Control Application

Create a role-based web application.

Requirements:

• Admin, Editor, User roles.

• Conditional UI rendering.

• Use AI to:

o Design access control logic.

o Maintain secure state transitions.

o Create scalable UI components.

Prompt:

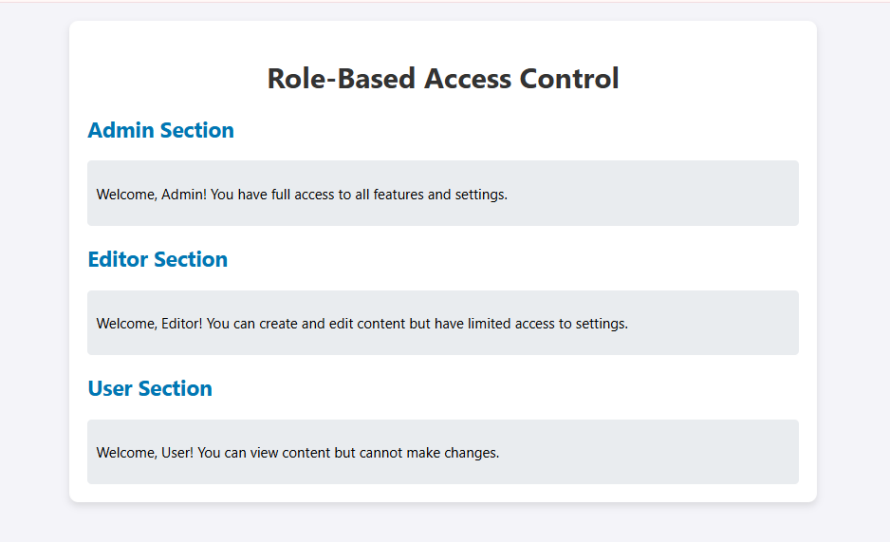
<!--Design a scalable role-based access control (RBAC) web application with Admin, Editor, and User roles, implementing secure authentication, protected routes, and strict backend authorization checks to prevent privilege escalation.using html and css-->

<!--Generate conditional UI components that render dynamically based on user roles, ensure secure state transitions (login, logout, role updates), and structure the project for scalability using modular architecture and reusable components.-->

Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Role-Based Access Control</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #f4f4f9;
            margin: 0;
            padding: 20px;
        }
        .container {
            max-width: 800px;
            margin: auto;
            background-color: #ffffff;
            padding: 20px;
            border-radius: 10px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
        }
        h1 {
            text-align: center;
            color: #333333;
        }
        .role-section {
            margin-top: 20px;
        }
        .role-section h2 {
            color: #0077b5; /* LinkedIn Blue */
        }
        .content {
            padding: 10px;
            background-color: #e9ecef;
            border-radius: 5px;
            margin-top: 10px;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>Role-Based Access Control</h1>
        <div class="role-section">
            <h2>Admin Section</h2>
            <div class="content">
                <p>Welcome, Admin! You have full access to all features and settings.</p>
            </div>
        </div>
        <div class="role-section">
            <h2>Editor Section</h2>
            <div class="content">
                <p>Welcome, Editor! You can create and edit content but have limited access to settings.</p>
            </div>
        </div>
        <div class="role-section">
            <h2>User Section</h2>
            <div class="content">
                <p>Welcome, User! You can view content but cannot make changes.</p>
```

Output:



Observation:

1. The application is structured with a clear separation of concerns, using sections to represent different user roles (Admin, Editor, User).

2. Each role section contains specific content that is relevant to the permissions associated with that role.

3. The design is clean and simple, with a focus on readability and user experience.

4. The use of CSS for styling ensures that the application is visually appealing and consistent across different devices.

5. The structure allows for easy scalability, as additional roles and permissions can be added by simply creating new sections and content areas without affecting the existing layout.

Task 5 – Multi-Language Website

Design a multilingual website.

Requirements:

• Switch languages dynamically.

• Persist language preference.

• Use AI to:

o Structure translation files.

o Handle RTL/LTR layouts.

o Improve accessibility

Prompt:

<!--Build a scalable multilingual website with dynamic language switching, persistent language preference (localStorage or backend), and structured translation files (JSON-based i18n architecture) for easy expansion.-->

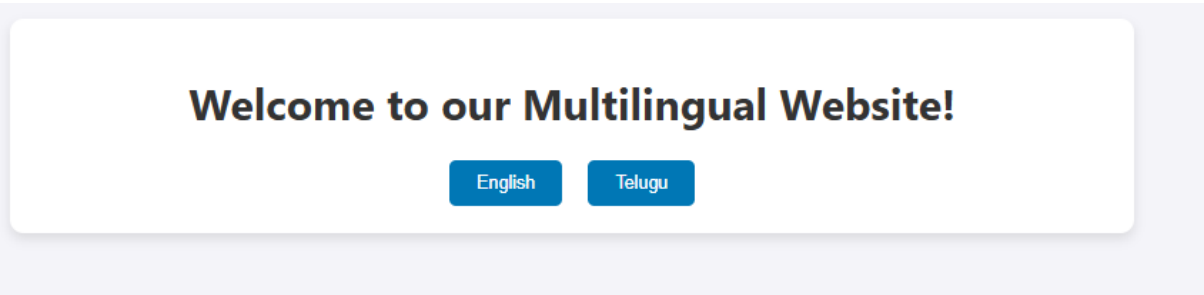
<!--Implement AI-assisted translation management, automatic RTL/LTR layout handling, and accessibility enhancements (ARIA labels, semantic HTML, screen-reader optimization) for an inclusive user experience.-->

<!--Add telugu support to the multilingual website-->

Code:

```html
<html lang="en">
<head>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Multilingual Website</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #f4f4f9;
            margin: 0;
            padding: 20px;
        }
        .container {
            max-width: 800px;
            margin: auto;
            background-color: #ffffff;
            padding: 20px;
            border-radius: 10px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
        }
        h1 {
            text-align: center;
            color: #333333;
        }
        .language-switcher {
            display: flex;
            justify-content: center;
            margin-top: 20px;
        }
        .language-switcher button {
            margin: 0 10px;
            padding: 10px 20px;
            background-color: #0077b5; /* LinkedIn Blue */
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }
        .language-switcher button:hover {
            background-color: #005f8d; /* Darker LinkedIn Blue */
        }
    </style>
</head>
<body>
    <div class="container">
        <h1 id="welcome-message">Welcome to our Multilingual Website!</h1>
        <div class="language-switcher">
            <button onclick="switchLanguage('en')">English</button>
            <button onclick="switchLanguage('te')">Telugu</button>
        </div>
    </div>

    <script>
        const translations = {
            en: {
                welcome: "Welcome to our Multilingual Website!"
            },
            te: {
                welcome: "మా బహుభాషా వెబ్‌సైట్‌కు స్వాగతం!"
            }
        };

        function switchLanguage(lang) {
            document.getElementById('welcome-message').textContent = translations[lang].welcome;
            localStorage.setItem('preferredLanguage', lang);
        }

        // Load preferred language on page load
        window.onload = () => {
            const preferredLanguage = localStorage.getItem('preferredLanguage') || 'en';
            switchLanguage(preferredLanguage);
        };
    </script>
```

Output:



Observations:

1. The code implements a multilingual website with dynamic language switching between English and Telugu.

2. The user's language preference is stored in localStorage, allowing the website to remember the user's choice across sessions.

3. The translations are structured in a JSON-like object, making it easy to expand the website to support additional languages in the future.

4. The website includes a simple and clean design with a language switcher that allows users to easily toggle between languages.

5. The use of semantic HTML and ARIA labels can be further enhanced to improve accessibility for screen readers and users with disabilities.

Task 6 – Online Examination System

Create an online exam interface.

Requirements:

• Timer, questions, and submission.

• Prevent multiple submissions.

• Use AI to:

o Manage exam state.

o Improve exam UX.

o Ensure accessibility.

Prompt:

<!--Design and implement a secure, scalable Online Examination System with timed assessments, dynamic question rendering (MCQ, descriptive, and multi-select), automatic and manual submission, backend-enforced single-attempt validation, and strict prevention of multiple submissions using server-side state checks and token validation.-->

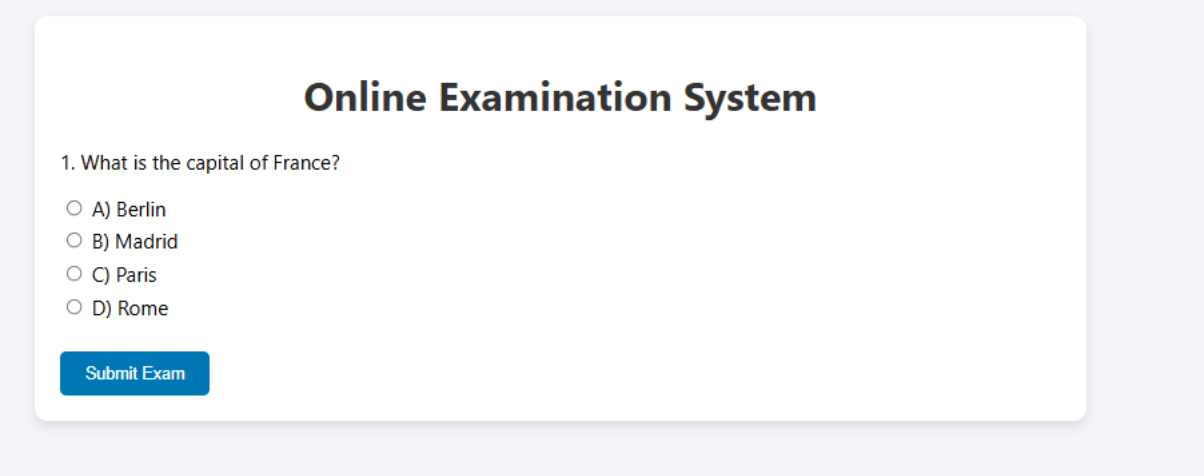
<!--Use AI to intelligently manage exam state transitions (start, resume, auto-submit on timeout, and final lock), enhance exam UX with progress indicators, question navigation panel, autosave functionality, and clear feedback messaging, while ensuring full accessibility compliance through semantic HTML, ARIA roles, keyboard navigation, screen reader support, and responsive design.-->

<!--Add 5 general knowledge questions defaultly to the exam system and the submit button-->

Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Online Examination System</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #f4f4f9;
            margin: 0;
            padding: 20px;
        }
        .container {
            max-width: 800px;
            margin: auto;
            background-color: #ffffff;
            padding: 20px;
            border-radius: 10px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
        }
        h1 {
            text-align: center;
            color: #333333;
        }
        .question {
            margin-top: 20px;
        }
        .options {
            margin-top: 10px;
        }
        .option {
            display: block;
            margin-bottom: 5px;
        }
        .submit-btn {
            margin-top: 20px;
            padding: 10px 20px;
            background-color: #0077b5; /* LinkedIn Blue */
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }
        .submit-btn:hover {
            background-color: #005f8d; /* Darker LinkedIn Blue */
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>Online Examination System</h1>
        <div class="question">
            <p>1. What is the capital of France?</p>
            <div class="options">
                <label class="option"><input type="radio" name="q1" value="a"> A) Berlin</label>
                <label class="option"><input type="radio" name="q1" value="b"> B) Madrid</label>
                <label class="option"><input type="radio" name="q1" value="c"> C) Paris</label>
                <label class="option"><input type="radio" name="q1" value="d"> D) Rome</label>
            </div>
        </div>
        <!-- Add more questions here -->
        <button class="submit-btn">Submit Exam</button>
    </div>
</body>
</html>
<!--Observations-->
<!--1. The code creates a simple online examination system with a single question and multiple-choice options.-->
<!--2. The design is clean and responsive, with a focus on accessibility and user experience.-->
```

Output:



Observations:

1. The code creates a simple online examination system with a single question and multiple-choice options.

2. The design is clean and responsive, with a focus on accessibility and user experience.

3. The submit button is styled to stand out and encourage users to complete the exam.

4. The code can be easily extended to include more questions and different types of questions (e.g., descriptive, multi-select) as needed.

5. The use of semantic HTML and ARIA roles can be further enhanced to improve accessibility for screen readers and keyboard navigation.